

Two-State Max-Plus Comparison Is Decidable

Keigo Oka*

September 1, 2026

Abstract

Given an arbitrary finite max-plus automaton A and a max-plus automaton B with at most two states, it is decidable whether $\llbracket A \rrbracket(w) \leq \llbracket B \rrbracket(w)$ for every word w . The proof uses a one-dimensional projective normal form for two-state dynamics. Outside an effective bounded region, a transition has one of three tail behaviors: it propagates the projective gap magnitude by a fixed shift with gap-independent height increment, forgets the gap with gap-independent height increment, or reads the gap magnitude into the height increment and then forgets it. In particular, any transition whose output depends on the unbounded gap destroys that gap. This yields an exact one-counter realization of B . Effective semilinearity of context-free Parikh images then reduces comparison to Presburger arithmetic. As a consequence, comparison, equivalence, and positivity for two-state max-plus automata are decidable.

1 Introduction

Let

$$\mathbb{Z}_{\max} = (\mathbb{Z} \cup \{-\infty\}, \max, +)$$

be the max-plus semiring. A finite max-plus automaton A over an alphabet Σ computes a series

$$\llbracket A \rrbracket : \Sigma^* \longrightarrow \mathbb{Z}_{\max}$$

by taking the maximum weight of an accepting run.

The exact comparison problem asks, for two automata A, B , whether

$$\llbracket A \rrbracket(w) \leq \llbracket B \rrbracket(w) \quad \text{for all } w \in \Sigma^*.$$

We prove the following.

Theorem 1.1 (Two-state right-hand comparison). *Let A be an arbitrary finite max-plus automaton over $\mathbb{Z}_{\max}$, and let B be a max-plus automaton with at most two states. It is decidable whether*

$$\llbracket A \rrbracket(w) \leq \llbracket B \rrbracket(w) \quad \text{for every } w \in \Sigma^*.$$

Corollary 1.2. *The following problems are decidable:*

- (i) *comparison of two max-plus automata with at most two states;*
- (ii) *equivalence of two max-plus automata with at most two states;*
- (iii) *positivity of a max-plus automaton with at most two states over any finite alphabet.*

*Email: ogiekako@gmail.com. ORCID: 0009-0007-8119-9267.

The proof uses one standard formal-language theorem. A two-state forward vector has only one unbounded projective coordinate, namely the difference of its two entries. For sufficiently large absolute difference, each letter has a fixed tail form. The essential property is one-way: if the current height increment depends on the magnitude of the unbounded projective gap, then the successor projective state no longer depends on that magnitude. A step that propagates the gap has a gap-independent height increment. This permits exact simulation by one nonnegative counter. The accepting transition language of the synchronized simulation is context-free, so effective Parikh semilinearity [1, 2] reduces the existence of a comparison violation to Presburger arithmetic.

We allow arbitrary max-plus initial and final weights and words in Σ^* .

No complexity bound is claimed here.

2 Two-state projective dynamics

If B has only one state, add a permanently dead second state; this leaves the computed series unchanged. Thus fix a two-state max-plus automaton B . Its forward row after reading a prefix is a vector

$$x = (x_1, x_2) \in \mathbb{Z}_{\max}^2.$$

When $x \neq (-\infty, -\infty)$, define its height

$$H(x) = \max(x_1, x_2).$$

If both coordinates are finite, define the signed projective gap

$$z(x) = x_1 - x_2 \in \mathbb{Z}.$$

If exactly one coordinate is finite, use $+\infty$ or $-\infty$ for the corresponding infinite gap. After subtracting the common height, every finite projective state is therefore represented by

$$(0, -n) \quad \text{or} \quad (-n, 0), \quad n \in \mathbb{N}.$$

The unbounded projective information is one-dimensional.

Let a letter σ act by

$$M = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \mathbb{Z}_{\max}^{2 \times 2}.$$

For a finite signed gap, use the representative $x = (z, 0)$. Direct max-plus multiplication gives

$$(xM)_1 = \max(z + a, c), \tag{1}$$

$$(xM)_2 = \max(z + b, d). \tag{2}$$

Hence the successor signed gap is

$$F_M(z) = \max(z + a, c) - \max(z + b, d), \tag{3}$$

whenever both displayed maxima are finite, with the evident infinite-gap conventions otherwise.

Write

$$A_M = \max(a, b), \quad C_M = \max(c, d).$$

Subtracting the input height gives the height increment

$$\delta_M(z) = \begin{cases} \max(A_M, C_M - z), & z \geq 0, \\ \max(A_M + z, C_M), & z \leq 0, \end{cases} \tag{4}$$

whenever the successor is nondead.

3 The tail trichotomy

Let E be the set of all finite transition, initial, and final weights of B . Choose

$$K = 1 + \max\{|\alpha - \beta| : \alpha, \beta \in E\}, \quad (5)$$

with $K = 1$ if E has fewer than two elements. Thus K strictly dominates every finite transition breakpoint, every fixed tail shift below, and the final readout breakpoint.

Consider first the positive tail $z > K$. Every maximum in (3) then has a forced branch. The four active-row cases are summarized in Table 1. The negative tail follows by exchanging the two current-state coordinates.

active row (a, b)	successor projective behavior	height increment	class
a, b finite	fixed gap $a - b$	constant $\max(a, b)$	forget
exactly one finite	translation, or infinite gap	constant	propagate or forget
$a = b = -\infty$	fixed by (c, d) , independent of z	$C_M - z$	read-and-forget

Table 1: Tail behavior for $z > K$. A translation has gap magnitude $n' = n + s$ for a fixed integer s (with the symmetric negative-tail form obtained by exchanging the current states).

For completeness, if a is finite then $z > |c - a|$ forces $z + a > c$; if $a = -\infty$, the first successor coordinate is c . The same statement holds for b, d . If both a, b are finite, both successor coordinates come from the active row, so the gap is the fixed value $a - b$ and $\delta_M(z) = \max(a, b)$. If exactly one of a, b is finite, the active successor coordinate is H plus a fixed weight. The other coordinate is either absent, giving an infinite gap, or comes from the inactive row and differs by $n + s$ for a fixed integer s ; in either case the height increment is constant. Finally, if $a = b = -\infty$, both successor coordinates come from the inactive row, so their projective class is independent of z while (4) gives $\delta_M(z) = C_M - z$.

Thus there is a genuine third possibility that should not be conflated with reading: a step may forget the gap without using its magnitude in the height increment.

Lemma 3.1 (Tail trichotomy). *For every projective state with finite gap magnitude $n = |z| > K$, every nondead letter transition has exactly one of the following forms, with constants determined by the letter and the tail side:*

- (i) propagate: *the successor has a finite gap whose magnitude depends affinely on the old magnitude, $n' = n + s$, for a fixed integer s (and a fixed successor side), while the height increment is a fixed constant c ;*
- (ii) forget: *the successor projective state is independent of n , while the height increment is a fixed constant c ;*
- (iii) read-and-forget: *the successor projective state is independent of n , while the height increment is $c - n$.*

Here propagate refers to preservation of the dependence on n : the shifted value $n' = n + s$ may be at most K for a particular input, in which case the successor lies in the bounded region. In particular, whenever the height increment depends on the unbounded gap magnitude, that magnitude is not propagated to the successor.

Proof. The positive-tail case is the case split above, and the negative-tail case is obtained by exchanging the current-state coordinates. The choice of K makes every relevant comparison strict and also gives $K > |s|$ for every propagate shift s . Moreover, in cases (ii) and (iii), the successor

projective state is either an infinite-gap state or a finite-gap state whose magnitude is strictly less than K . Indeed, every finite successor gap in those cases is the difference of two finite entries of the letter matrix, so its absolute value is $< K$ by the definition of K . $\square$

Remark 3.2. *For example, the all-zero matrix has a silent forget tail: for every sufficiently large gap, the successor gap is 0 and the height increment is 0. This is why the useful structural statement is not a literal retain/read dichotomy, but the prohibition on simultaneously reading an unbounded magnitude into the output and propagating that magnitude to the future.*

Remark 3.3. *The statement is specific to the one-dimensional projective state of two states. With three states, two independent projective differences can coexist, and an update can read one combination while retaining another.*

4 Exact compilation to one counter

Lemma 4.1 (Functional one-counter realization). *From B one can effectively construct a one-counter transducer T_B such that, for every word w :*

- (i) *if $\llbracket B \rrbracket(w) = -\infty$, then T_B has no successful computation on w ;*
- (ii) *if $\llbracket B \rrbracket(w) \neq -\infty$, then T_B has a successful computation on w , and every successful computation on w emits exactly $\llbracket B \rrbracket(w)$.*

The construction may be chosen deterministic. Each input letter triggers a finite-control routine built from unit counter increments, blocking decrements, zero tests, and fixed integer emissions. Such a routine has a unique finite execution from any reachable configuration, although its running length may depend on the counter value (for example when the counter is drained to zero).

Proof. Use finite-control states B_z for every integer $-K \leq z \leq K$, infinite-gap states I_+ and I_- , and tail states T_+ and T_- . Their meanings are as follows. In B_z the exact signed projective gap is z ; in I_+ (respectively I_-) only the first (respectively second) coordinate is finite; and in T_+ or T_- the gap is finite with the indicated sign and magnitude $n > K$.

The counter invariant is:

$$\begin{array}{ll} \text{in } B_z, I_+, I_- & : \text{the counter is } 0, \\ \text{in } T_+, T_- & : \text{the counter is exactly } n = |z| > K. \end{array}$$

In addition, at every boundary between these finite-control routines, the total emission produced so far is exactly the height of the current forward vector.

If the initial forward vector is $(-\infty, -\infty)$, take T_B to have no successful computation, and there is nothing to prove. Otherwise its height is defined; the initial forward vector contributes that height as a fixed emission and initializes the invariant.

If the current gap is bounded, the exact successor and height increment are obtained from a finite lookup table. If the lookup successor is a tail gap of magnitude $n' > K$, load this fixed value from zero by n' unit increments; otherwise the counter remains zero. Because there are only finitely many bounded control states and letters, each such load is an effectively constructible finite routine. Its explicit size may depend on the numerical magnitudes of the weights; this is immaterial here because no complexity bound is claimed. If the current gap is infinite, the only surviving current coordinate determines the successor by another finite lookup. Suppose now that the counter stores a tail magnitude $n > K$. Apply Lemma 3.1.

In a propagate case, emit the fixed constant c and change the counter by the fixed shift s , using $|s|$ unit increments or blocking decrements. Since $n > K > |s|$, these decrements cannot block. Let $m = n + s > 0$ be the resulting counter value.

To determine whether $m \leq K$, use a deterministic destructive probe. For $j = 0, 1, \dots, K$, after exactly j probe decrements, test whether the counter is zero. If it is, then $m = j$; record that exact bounded gap in finite control and leave the counter at zero. If no zero test succeeds for $j \leq K$, perform one further decrement and then exactly $K + 1$ increments. This restores the original value m , including in the boundary case $m = K + 1$, and the machine remains in the appropriate tail control state. All probe steps emit zero.

In a forget case, the old magnitude will never be needed again. Drain the counter to zero with zero emission, emit the fixed constant c , and move to the successor projective control state. By the final observation in the proof of Lemma 3.1, that successor is either an infinite-gap state or a bounded finite-gap state, so counter value zero is the correct representation.

In a read-and-forget case, drain the counter to zero while emitting -1 per decrement, then emit the fixed constant c and move to the successor projective control state. The total contribution of the routine is exactly $c - n$. Again, the successor is bounded or infinite by Lemma 3.1, so the counter-zero invariant is restored. Dead transitions reject.

The final vector is handled by the same one-dimensional argument. Bounded and infinite projective states are finite-control lookups. Suppose, for example, that the current state is T_+ , so the forward vector is $(H, H - n)$ with $n > K$, and let the final weights be f_1, f_2 .

If f_1 is finite, then whenever f_2 is also finite,

$$n > K > |f_1 - f_2|$$

implies $f_1 > f_2 - n$. Hence the first coordinate necessarily realizes the final maximum, the final contribution is the fixed constant f_1 , and the counter may be drained silently. If $f_1 = -\infty$ and f_2 is finite, the final contribution is $f_2 - n$; drain the counter while emitting -1 per decrement and then emit f_2 . If both final weights are $-\infty$, reject. The case T_- is symmetric.

All routines above are deterministic: at each zero test the zero and positive-counter cases are disjoint, and every other step is forced by the current finite control and input letter.

We prove correctness by induction on the processed input prefix. At each boundary between letter routines, the finite control and counter encode the exact projective class of the forward vector as specified above, and the total emission produced so far is exactly its height. The bounded, infinite, propagate, forget, and read-and-forget cases above preserve this invariant, while a dead max-plus successor has no successful machine continuation. The initialization establishes the invariant for the initial forward vector, and the final routine adds exactly the remaining final contribution. Consequently, if $\llbracket B \rrbracket(w) = -\infty$ there is no successful computation, and otherwise the unique successful computation emits exactly $\llbracket B \rrbracket(w)$. $\square$

5 Deciding comparison

Proof of Theorem 1.1. First separate support. For a max-plus automaton, the set of words on which the value is finite is regular: ignore weights and retain exactly the finite-weight transitions, initial states, and final states. Hence it is decidable whether there exists a word with $\llbracket A \rrbracket(w)$ finite and $\llbracket B \rrbracket(w) = -\infty$. Any such word is an immediate violation.

Henceforth discard every $-\infty$ -weighted transition of A , and in the explicit start/end encoding retain only finite initial and final weights. This does not change the series computed by A : every run using a $-\infty$ weight has total weight $-\infty$, while a word with no remaining accepting run still has value $-\infty$. Thus every accepting run considered below has an ordinary integer weight.

Assume now that we work on the common finite support. Construct the functional one-counter transducer T_B from Lemma 4.1 and view its finite-control routines at the level of their unit one-counter transitions; counter-dependent loops are represented by ordinary loops in this finite transition graph and are not unrolled to a fixed length. Synchronize each transition that consumes a letter

σ with a finite-weight transition of A carrying the same letter, while internal counter transitions leave the A -state unchanged. Encode finite initial and final weights of A by dedicated start and end transitions. The resulting finite one-counter machine nondeterministically chooses an accepting run ρ of A , while the T_B component has only the exact output $\llbracket B \rrbracket(w)$ on the synchronized word.

Give every transition e of this finite product one-counter machine its own fresh symbol γ_e . Let L be the language of fresh-symbol sequences labeling successful computations. Then L is accepted by an effectively constructible ordinary one-counter automaton, hence by a pushdown automaton with one stack symbol above a bottom marker: increment pushes that symbol, blocking decrement pops it, and a zero test checks the bottom marker. Therefore an effective context-free grammar for L can be constructed. By the constructive form of Parikh's theorem [1, 2], its Parikh image

$$S = \Psi(L) \subseteq \mathbb{N}^m$$

is effectively semilinear.

Enumerate the product transitions as $e_1, \dots, e_m$. For each e_i , let $a_i \in \mathbb{Z}$ be its contribution to the chosen A -run weight (0 on internal transducer transitions), and let $b_i \in \mathbb{Z}$ be its transducer emission. The dedicated start/end transitions include all finite initial and final contributions. Hence for every successful trace $t \in L$, with $v = \Psi(t)$,

$$\text{wt}_A(\rho_t) = \sum_{i=1}^m a_i v_i, \quad \llbracket B \rrbracket(w_t) = \sum_{i=1}^m b_i v_i,$$

where w_t and ρ_t are respectively the synchronized input word and the accepting run of A represented by t . The second equality uses Lemma 4.1.

We claim that, after the support check above, a comparison violation exists if and only if

$$\exists v \in S : \sum_{i=1}^m (a_i - b_i) v_i > 0. \quad (*)$$

For the forward implication, suppose $\llbracket A \rrbracket(w) > \llbracket B \rrbracket(w)$. The support check ensures that $\llbracket B \rrbracket(w)$ is finite. Since w has only finitely many runs in the finite automaton A , some accepting run ρ attains $\llbracket A \rrbracket(w)$. Synchronizing ρ with the successful computation of T_B on w gives a trace $t \in L$. Its Parikh vector $v = \Psi(t)$ satisfies $(*)$.

Conversely, suppose $v \in S$ satisfies $(*)$. By the definition of $S = \Psi(L)$, there exists an actual successful trace $t \in L$ with $\Psi(t) = v$. This trace represents some word w_t and accepting run ρ_t of A , and $(*)$ gives

$$\text{wt}_A(\rho_t) > \llbracket B \rrbracket(w_t).$$

Therefore

$$\llbracket A \rrbracket(w_t) \geq \text{wt}_A(\rho_t) > \llbracket B \rrbracket(w_t),$$

so w_t is a genuine comparison violation.

Finally, because S is effectively semilinear, condition $(*)$ is an existential Presburger condition over an effectively given semilinear set and is decidable. Hence it is decidable whether any comparison violation exists. $\square$

Proof of Corollary 1.2. Comparison follows immediately when both automata have at most two states. Equivalence is the conjunction of the two comparison directions. For positivity, let A_0 be the one-state automaton with constant value zero. Then

$$\llbracket A_0 \rrbracket \leq \llbracket B \rrbracket$$

if and only if $\llbracket B \rrbracket(w) \geq 0$ for every word w . $\square$

6 Implementation check

The companion script `checks/check_two_state_maxplus.py` checks the projective formulas, exhaustively classifies a finite family of letter tails into propagate / forget / read-and-forget, and compares direct and projective evaluators on finite test families. The theorem rests on the proof above, not on finite enumeration.

Research provenance

This is an unrefereed preprint. AI systems materially assisted in finding and drafting the argument and in writing the code. The written proof should be checked on its own; model agreement and the finite regression are not independent mathematical evidence.

References

- [1] Rohit J. Parikh. On Context-Free Languages. *Journal of the ACM*, 13(4):570–581, 1966.
- [2] Eryk Kopczyński and Anthony Widjaja To. Parikh Images of Grammars: Complexity and Applications. In *LICS 2010*, pp. 80–89, 2010. DOI: 10.1109/LICS.2010.21.